

Research Brief: Parallel Training Decomposition for Neural MAC Polar Decoders

Your Mission

You are a deep learning and information theory researcher. I need you to do a **deep, creative research exploration** to find a way to parallelize the training of a neural decoder for polar codes on the **two-user Multiple Access Channel (MAC)**. This is currently an open problem that, if solved, would be a major breakthrough.

I will explain the problem in full detail below. Your job is to: 1. Understand why the current approach requires $O(N \log N)$ sequential operations 2. Understand why the single-user solution (`fast_ce`) doesn't work for MAC 3. **Search deeply** for mathematical decompositions, algebraic tricks, or alternative training paradigms that could reduce the sequential depth to $O(\log N)$ or $O(N)$ 4. Look at related problems in other fields (group theory, tensor decompositions, parallel algorithms, tropical algebra, etc.) 5. Propose concrete approaches we can implement and test

Part 1: Background — What Works for Single-User (NPD)

The Single-User SC Decoder

For a single-user binary-input channel $W(y|x)$, the SC decoder operates on a binary tree of depth $n = \log_2(N)$. At each internal node, two operations are performed:

f-node (check node):

$$L_{\text{out}} = \text{sign}(L_a) * \text{sign}(L_b) * \min(|L_a|, |L_b|)$$

where L_a, L_b are LLR (log-likelihood ratio) values from children.

g-node (variable node):

$$L_{\text{out}} = L_b + (-1)^u * L_a$$

where u is the decided bit from the f-node.

How NPD (Neural Polar Decoder) Achieves $O(\log N)$ Training

Aharoni et al. (2024) replace f and g with neural networks. The key insight for fast training:

At each tree depth d ($d = 0, 1, \dots, n-1$): - There are $N/2^{(d+1)}$ independent f-node computations - There are $N/2^{(d+1)}$ independent g-node computations - These can ALL be computed in parallel (they don't depend on each other)

The trick — “fast cross-entropy” (`fast_ce`):

During training with teacher forcing, the true bits are known. Therefore: 1. At depth d , ALL f-nodes at that depth can be computed simultaneously 2. Given the f-node outputs and the TRUE bits, ALL g-nodes at that depth can be computed simultaneously 3. Move to depth $d+1$ and repeat

This gives **$n = \log_2(N)$ sequential steps** instead of $2N-1$ sequential leaf decisions. For $N=1024$, that's 10 steps instead of 2047.

Why this works mathematically: - The f-node operation $\text{sign}(a) * \text{sign}(b) * \min(|a|, |b|)$ is a function of only (L_a, L_b) — no dependence on other positions - The g-node operation $L_b + (-1)^u * L_a$ depends on u (the decided bit), but during teacher forcing, u is KNOWN - Therefore, at each depth, all operations are independent given the layer above and the true bits

The Neural Architecture for fast_ce

The NN replaces f and g with:

```
f_nn(L_a, L_b) = MLP([L_a, L_b])      # 2 inputs → 1 output
g_nn(L_a, L_b, u) = MLP([L_a, L_b, u]) # 3 inputs → 1 output
```

During fast_ce training: - Depth 0: compute all $N/2$ f-nodes in parallel $\rightarrow (N/2,)$ outputs - Given true bits: compute all $N/2$ g-nodes in parallel $\rightarrow (N/2,)$ outputs - Depth 1: compute all $N/4$ f-nodes in parallel $\rightarrow (N/4,)$ outputs - ... and so on for $\log_2(N)$ depths

The loss is cross-entropy at each leaf (depth n), and gradients flow back through only $n = \log_2(N)$ sequential operations.

This is the key advantage of NPD over our approach.

Part 2: The MAC Problem — Why fast_ce Fails

The Two-User MAC SC Decoder

For the two-user MAC $W(z|x,y)$, the SC decoder operates on **2x2 probability tensors** instead of scalar LLRs. Each tensor $P[u,v]$ represents the joint probability of (user_U_bit, user_V_bit).

The three tree operations are:

CalcLeft (circular convolution):

$$P_{\text{left}}[u, v] = \sum_{\{u', v'\}} P_{\text{parent}}[u u', v v'] * P_{\text{right}}[u', v']$$

Written out for the 2x2 case:

$$\begin{aligned} P_{\text{left}}[0,0] &= P_{\text{parent}}[0,0]*P_{\text{right}}[0,0] + P_{\text{parent}}[0,1]*P_{\text{right}}[0,1] + P_{\text{parent}}[1,0]*P_{\text{right}}[1,0] + P_{\text{parent}}[1,1]*P_{\text{right}}[1,1] \\ P_{\text{left}}[0,1] &= P_{\text{parent}}[0,1]*P_{\text{right}}[0,0] + P_{\text{parent}}[0,0]*P_{\text{right}}[0,1] + P_{\text{parent}}[1,1]*P_{\text{right}}[1,0] + P_{\text{parent}}[1,0]*P_{\text{right}}[1,1] \\ P_{\text{left}}[1,0] &= P_{\text{parent}}[1,0]*P_{\text{right}}[0,0] + P_{\text{parent}}[1,1]*P_{\text{right}}[0,1] + P_{\text{parent}}[0,0]*P_{\text{right}}[1,0] + P_{\text{parent}}[0,1]*P_{\text{right}}[1,1] \\ P_{\text{left}}[1,1] &= P_{\text{parent}}[1,1]*P_{\text{right}}[0,0] + P_{\text{parent}}[1,0]*P_{\text{right}}[0,1] + P_{\text{parent}}[0,1]*P_{\text{right}}[1,0] + P_{\text{parent}}[0,0]*P_{\text{right}}[1,1] \end{aligned}$$

This is a **circular convolution over $\mathbb{Z}_2 \times \mathbb{Z}_2$** (the Klein four-group).

CalcRight (conditional product):

$$P_{\text{right}}[u, v] = P_{\text{parent}}[d_u u, d_v v] * P_{\text{left}}[d_u, d_v]$$

where (d_u, d_v) are the decided bits from the left child.

CalcParent (inverse convolution / marginalization):

$$P_{\text{parent}}[u, v] = \sum_{\{u', v'\}} P_{\text{left}}[u u', v v'] * P_{\text{right}}[u', v']$$

Why fast_ce Doesn't Work for MAC

Problem 1: The decoding path is not level-aligned

For the single-user decoder, leaf decisions follow a fixed pattern: at depth d , positions 0, 2, 4, ... are f-nodes and positions 1, 3, 5, ... are g-nodes. This is the same at every depth.

For the MAC decoder with Class B path (interleaved), the decoding order is:

$b = [0, 1, 0, 1, 0, 1, \dots, 0, 1]$ (alternating User U and User V)

The $2N$ leaf decisions don't correspond to tree depth levels. Leaf decision at step t may require navigating UP the tree (CalcParent) before going DOWN (CalcLeft/CalcRight), creating cross-level dependencies.

Problem 2: CalcLeft is a 4-element coupled operation

In the single-user case:

$f(L_a, L_b) \rightarrow \text{single scalar}$

The output is a single number. Two f-nodes at the same depth are independent.

In the MAC case:

$\text{CalcLeft}(P_{\text{parent}}, P_{\text{right}}) \rightarrow 2 \times 2 \text{ tensor (4 coupled values)}$

The 4 output values are ALL functions of ALL 4 input values from P_{parent} and ALL 4 from P_{right} . They cannot be decomposed into independent per-element operations.

Problem 3: CalcParent creates bottom-up dependencies

In the single-user decoder, information flows strictly top-down during decoding. There is no bottom-up CalcParent operation.

In the MAC decoder (Class B), CalcParent is called when the decoder needs to "go up" the tree to reach the next leaf. CalcParent takes the LEFT and RIGHT child edges and combines them to produce the PARENT edge. This creates a dependency: the parent edge depends on decisions made in both subtrees.

What We Tried (and Failed)

Attempt 1: Treat (u,v) as 4-class output, use sign-encoding like NPD

Map the 4-class joint (u,v) to a sign vector: $(u,v) \rightarrow \{(-1,-1), (-1,+1), (+1,-1), (+1,+1)\}$.

Then try to decompose CalcLeft as:

$\text{output}[i] = f_{\text{nn}}(\text{input}[i] * \text{sign_encoding})$

Result: Loss plateaus at 0.30 (near random for 4 classes). The sign encoding doesn't capture the circular convolution structure.

Attempt 2: Walsh-Hadamard Transform (WHT) decomposition

The circular convolution over $Z^2 \times Z^2$ can be diagonalized by the WHT:

$\text{WHT} = 1/2 * \begin{bmatrix} [1, 1, 1, 1], & [1, -1, 1, -1], & [1, 1, -1, -1], & [1, -1, -1, -1] \end{bmatrix}$

In the WHT domain, circular convolution becomes element-wise multiplication:

$\text{WHT}(P_{\text{left}}) = \text{WHT}(P_{\text{parent}}) \quad \text{WHT}(P_{\text{right}})$

We tried training in the WHT domain:

Input: $\text{WHT}(\text{P_parent}), \text{WHT}(\text{P_right}) \rightarrow 4+4=8$ values

Output: $\text{WHT}(\text{P_left}) \rightarrow 4$ values

Target: element-wise product of WHT inputs

Result: Loss plateaus at 0.29. The WHT diagonalizes the circular convolution, but the neural network must also handle: - Log-domain operations (the actual tensors are in log-probability space) - CalcRight (which is NOT a circular convolution) - The interleaved decision order

The WHT trick only helps CalcLeft, not the full pipeline.

Attempt 3: One-hot encoding with residual

Encode (u,v) as 4-dimensional one-hot vector. Use residual connections matching the analytical formula.

Result: Same plateau at 0.30.

Part 3: The Core Mathematical Challenge

What We Need

We need a representation where, during teacher-forced training:

1. **At each tree depth d** , all CalcLeft operations at depth d can be computed **independently** (given the level above)
2. **Given the true bits**, all CalcRight operations at depth d can be computed **independently**
3. **CalcParent can be avoided** or computed in a way that doesn't create cross-depth dependencies

Why This Is Hard

The key obstacle is that CalcLeft is a **circular convolution over $\mathbb{Z}_2 \times \mathbb{Z}_2$** , not a simple per-element operation. In the frequency domain (WHT), it becomes element-wise multiplication, but:

- We work in **log domain** (log-probabilities, not probabilities)
- In log domain, element-wise multiplication $\rightarrow$ element-wise addition
- But the WHT itself involves sums, which in log domain are logsumexp operations
- logsumexp is not separable per-element

So the chain is:

Log-domain tensor $\rightarrow \exp \rightarrow \text{WHT} \rightarrow \text{element-wise multiply} \rightarrow \text{inverse WHT} \rightarrow \log$

This is not decomposable into independent per-element operations.

The CalcParent Problem

Even if CalcLeft/CalcRight could be parallelized, CalcParent creates vertical dependencies. In the single-user decoder, there is no CalcParent — information flows only top-down. In the MAC decoder (Class B), CalcParent is needed because:

- The decoding path alternates between User U and User V

- After deciding a User U bit in the left subtree, the decoder needs to “go up” to reach a User V bit in the right subtree
- Going up requires CalcParent to combine the left subtree’s information with the right subtree

For **Class A** (path $0^N 1^N$ — all V first, then all U) and **Class C** (path $1^N 0^N$ — all U first, then all V), CalcParent is NOT needed because the decoder processes each subtree completely before moving to the other. For these extreme paths, the decoder reduces to two independent single-user decoders, and NPD-style parallelization WOULD work.

But Class A and C cannot achieve the **symmetric rate point** ($R_u = R_v$). Only Class B (and other interleaved paths) can, and these require CalcParent.

Part 4: Directions to Explore

Here are starting points for your research. Please explore these AND any other ideas you find.

Direction 1: Algebraic Decomposition of Circular Convolution over $\mathbb{Z}_2 \times \mathbb{Z}_2$

The circular convolution over the Klein four-group $V_4 = \mathbb{Z}_2 \times \mathbb{Z}_2$ has special structure: - V_4 is abelian $\rightarrow$ Fourier transform exists (the WHT) - V_4 has exactly 4 characters: $\chi_{00}, \chi_{01}, \chi_{10}, \chi_{11}$ - In the character domain, convolution = pointwise product

Question: Can we find a neural representation that operates in the character domain (WHT domain) where CalcLeft is element-wise, and handle the log-domain issue through learned nonlinearities?

Specifically: if we train the z -encoder to output embeddings in a “frequency-like” domain where CalcLeft decomposes, can the neural network learn this decomposition end-to-end?

Direction 2: Tropical Algebra / Min-Plus Semiring

In log domain, probability multiplication $\rightarrow$ addition, probability addition $\rightarrow$ logsumexp.

The **tropical semiring** (min, +) or (max, +) replaces logsumexp with max. This IS decomposable:

$$\max(a + c, b + d) \neq \max(a, b) + \max(c, d) \quad [\text{not decomposable in general}]$$

But: if we approximate logsumexp $\approx$ max (the “max-log” approximation commonly used in LDPC decoders), does CalcLeft become decomposable?

Question: Under the max-log approximation, can the circular convolution be decomposed into independent per-element operations?

Direction 3: Factored Belief Propagation / Message Passing

Instead of the SC tree walk, consider a **factor graph** representation of the MAC polar code. Belief propagation on factor graphs can be parallelized: - All variable-to-factor messages can be computed in parallel - All factor-to-variable messages can be computed in parallel - This gives $O(\text{iterations} \times 2)$ sequential depth, independent of N

Question: Can we replace the SC tree walk with a learned BP-like message passing scheme that operates on the same polar code structure but with parallel message updates?

Related work: Nachmani et al. (2018) used weighted BP for LDPC codes with learned weights. Can this be adapted for MAC polar codes?

Direction 4: Attention / Transformer-Based Parallel Decoder

Transformers process all positions in parallel with $O(1)$ sequential depth (per layer). A transformer-based decoder could: - Input: all N channel observations - Output: all $2N$ bit decisions simultaneously - Training: fully parallel

Question: Can a transformer learn the SC decoding function for MAC polar codes? The challenge is that SC is inherently sequential — later decisions depend on earlier ones. But with teacher forcing, the true bits are known, so all decisions could potentially be made in parallel.

Related: TransCoder (2025) applies transformers to enhance polar code decoding. Could a similar approach work for MAC?

Direction 5: Depth-Parallel Training with Auxiliary Losses

Instead of full end-to-end training through the sequential tree walk, train each depth independently:

- Depth 0: train CalcLeft_0 to approximate analytical CalcLeft given root embeddings $\rightarrow$ parallel over $N/2$ positions
- Depth 1: train CalcLeft_1 given depth-0 outputs $\rightarrow$ parallel over $N/4$ positions
- ...

Each depth's training is fully parallel. The challenge is defining proper targets for intermediate depths.

Question: Can we define meaningful training targets at each depth that, when composed, yield a good end-to-end decoder?

Direction 6: Two-Phase Decomposition for Class B

Class B path alternates U and V decisions. What if we decompose it into two phases: - Phase 1: Decode all U bits (ignoring V), using a single-user decoder for User U - Phase 2: Decode all V bits conditioned on the (possibly incorrect) U decisions

Each phase is a single-user decode and can use `fast_ce`!

Problem: Phase 1 will be suboptimal because U operates above marginal capacity ($R_u = 0.48 > I(Z;X) = 0.46$). But if Phase 1 gives a “good enough” approximation, Phase 2 can correct.

Question: Can we use iterative refinement? Phase 1 $\rightarrow$ Phase 2 $\rightarrow$ re-decode U given $V \rightarrow$ re-decode V given $U \rightarrow \dots$ converging to the joint optimum?

Direction 7: Differentiable Relaxation of the Tree Walk

Instead of the exact sequential tree walk, consider a **relaxed** version where: - All $2N$ leaf embeddings are computed simultaneously (in parallel) - A learned attention mechanism models the sequential dependencies - Training uses a differentiable relaxation of the hard sequential decisions

Question: Can we approximate the sequential tree walk with a fixed-depth ($O(\log N)$) attention network that captures the essential dependencies?

Direction 8: Group-Theoretic Approaches

The MAC polar code structure is built on the group $Z_2 \times Z_2$. The polarization transform $F^{\{\square n\}}$ acts on this group through tensor products.

Question: Are there known results in computational group theory about parallelizing convolutions over finite groups? Specifically, for the group Z_2^k (which generalizes $Z_2 \times Z_2$), are there sub-linear depth circuits for computing circular convolutions?

Direction 9: Separation of CalcLeft into User-Specific Components

CalcLeft over $Z_2 \times Z_2$ can be viewed as two coupled Z_2 convolutions:

$$P_{\text{left}}[u, v] = \sum_{\{u', v'\}} P_{\text{parent}}[u' u', v v'] * P_{\text{right}}[u', v']$$

Can we decompose this as:

$$P_{\text{left}}[u, v] = f_U(P_{\text{parent}}[\cdot, v], P_{\text{right}}[\cdot, v]) * f_V(P_{\text{parent}}[u, \cdot], P_{\text{right}}[u, \cdot])$$

i.e., approximate the joint convolution as a product of marginal convolutions?

If so, each marginal convolution is a Z_2 convolution (just like single-user!) and can be parallelized using NPD's approach.

Question: Under what conditions does this factored approximation work? What is the approximation error? Can the neural network learn to compensate for the error?

Direction 10: Curriculum of Parallelism

Start with a fully sequential decoder (current approach). Gradually “parallelize” during training: - Phase 1: Train sequentially (current approach, gets good weights) - Phase 2: Group adjacent leaves and process them in parallel (2x speedup, some approximation) - Phase 3: Group larger blocks (4x, 8x, ... speedup)

Each phase starts from the previous phase's weights and adapts to the increased parallelism.

Question: Can the neural network learn to compensate for the approximation introduced by processing non-independent operations in parallel?

Part 5: Key Constraints

Whatever solution you propose must satisfy:

1. **Correct at convergence:** The decoder must match or approach analytical SC performance
2. **$O(\log N)$ or $O(N)$ sequential depth during training** (currently $O(N \log N)$)
3. **Work for Class B paths** (interleaved decoding, $R_u \approx R_v$)
4. **Compatible with teacher forcing** (true bits available during training)
5. **Scalable to $N=256, 512, 1024$** (current approach fails at $N \geq 256$ on continuous channels)

Approximate solutions are welcome — even an $O(N)$ sequential depth (vs current $O(N \log N)$) would be valuable.

Part 6: Key References

1. **Aharoni et al. (2024)** — “Data-Driven Neural Polar Codes for Unknown Channels With and Without Memory” — The NPD paper that introduces fast_ce for single-user. IEEE ISIT 2024.
 2. **Ren et al. (2025)** — “Successive Cancellation Decoding of Polar Codes for the Two-User MAC Using Computational Graphs” — The MAC SC decoder we base our work on. Defines CalcLeft/CalcRight/CalcParent.
 3. **Sasoglu et al. (2013)** — “Polar Codes for the Two-User Multiple-Access Channel” — Theoretical foundation. Proves polar codes achieve MAC capacity.
 4. **Arikan (2009)** — “Channel Polarization” — Original polar codes paper.
 5. **Hebbar et al. (2023)** — “CRISP: Curriculum Based Sequential Neural Decoders for Polar Code Family” — Alternative neural decoder approach. ICML 2023.
 6. **Nachmani et al. (2018)** — “Deep Learning Methods for Improved Decoding of Linear Codes” — Weighted BP with learned parameters.
 7. **Our comprehensive report** — Available separately. Contains full architecture details, results, and failed approaches.
-

Part 7: What a Useful Answer Looks Like

I am looking for:

1. **Specific mathematical decompositions** that reduce sequential depth — not vague suggestions, but concrete formulas or algorithms I can implement
2. **Pointers to relevant literature** I may have missed — papers on parallel algorithms for group convolutions, tropical geometry, tensor network decompositions, etc.
3. **Feasibility analysis** — for each proposed approach, estimate: (a) expected approximation error, (b) implementation complexity, (c) whether it’s been tried in related contexts
4. **A ranked list** of approaches by likelihood of success, with your reasoning
5. **Pseudocode** for the most promising approach, showing exactly how the parallel training would work

I do NOT need: - Vague suggestions like “use transformers” without explaining how to handle the sequential dependencies - Approaches that only work for Class A/C (we already know those reduce to single-user) - Approaches that require abandoning the polar code structure entirely

Summary of the Core Problem in One Sentence

Find a way to decompose the circular convolution over $\mathbb{Z}_2^N \times \mathbb{Z}_2^N$ (CalcLeft) and its inverse (CalcParent) in log-probability domain such that, during teacher-forced training, all operations at each tree depth can be computed independently — reducing training depth from $O(N \log N)$ to $O(\log N)$.